

# Healthcare Clinic & Patient Appointment System

A Modular, Conflict-Free Clinical Scheduling & Emergency Triage System in Java

|                     |                                                      |                      |                                                 |
|---------------------|------------------------------------------------------|----------------------|-------------------------------------------------|
| Student / Author:   | Krish Kumar (Reg No: 25BAI10528)                     | Course / Platform:   | Object-Oriented Programming in Java (VITYarthi) |
| Email / GitHub:     | krish.25bai10528@vitbhopal.ac.in   @krishkumarsharma | Institution:         | VIT Bhopal University                           |
| Date of Submission: | September 2026                                       | Domain:              | Healthcare Operations & Clinical Scheduling     |
| Language & Runtime: | Java SE 17+ (Tested on OpenJDK 21)                   | Architecture & Data: | Three-Tier Layered / RFC 4180 CSV Persistence   |

## 1. Introduction & Problem Statement

Outpatient clinics and community health centers process numerous daily patient visits, doctor consultations, and emergency reviews. Manual record keeping or generic spreadsheets commonly lead to critical issues: **(1) Physician double-booking** where doctors are booked simultaneously in identical time slots, **(2) Delayed emergency triage** where patients with acute symptoms must wait behind routine consultations, **(3) Session data loss** upon program termination, and **(4) Console input crashes** caused by unhandled Scanner buffer desynchronization.

**Technical Solution:** This project delivers a fully standalone, dependency-free Java CLI application that provides: patient demographic intake, clinical urgency classification (CRITICAL → HIGH → MEDIUM → LOW), conflict-free appointment booking with physician availability checks, automated billing invoice computation, and multi-entity CSV persistence across application runs.

## 2. Functional & Non-Functional Requirements

### 2.1 Three Core Functional Modules

| Module                       | Feature                          | Functional Description                                                                                                                      |
|------------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Module 1<br>Patient & Triage | Patient Intake & Triage Priority | Registers patient demographics (age, gender, blood group) with urgency levels (CRITICAL, HIGH, MEDIUM, LOW) and automated priority queuing. |
| Module 2<br>Doctor Catalog   | Staff Directory & Search         | Maintains doctor profiles with consultation fees and available clinic days; supports keyword-based specialization search (e.g. Cardiology). |
| Module 3<br>Appointments     | Booking & Invoicing Engine       | Conflict-free appointment scheduling (rejects double-booking), visit completion notes, and itemized billing invoice calculation.            |
| Persistence                  | Multi-CSV Synchronization        | Automatic synchronization of all records to data/patients.csv, data/doctors.csv, and data/appointments.csv upon every update.               |

### 2.2 Non-Functional Requirements

- Performance:** O(1) instantaneous lookups using in-memory LinkedHashMap primary key indexing.
- Reliability:** Defensively traps NumberFormatException and stream EOF conditions without uncaught exceptions.
- Referential Integrity:** Rejects appointment bookings for non-existent patients or doctors.
- Zero Dependencies:** Built exclusively with standard Java SE libraries (java.util, java.io, java.util.regex).

## 3. System Architecture & Object-Oriented Design

The project follows a **Three-Tier Layered Architecture** with clear separation of concerns across 10 modular classes and enums:

| Class / Enum | Role / OOP Principle | Key Methods & Responsibilities                                                    |
|--------------|----------------------|-----------------------------------------------------------------------------------|
| Person.java  | Abstract Base Class  | Encapsulates common fields (id, name, phone, email); declares abstract getRole(). |
| Patient.java | Inheritance (Person) | Extends Person; adds age, gender, bloodGroup, and TriageSeverity classification.  |
| Doctor.java  | Inheritance (Person) | Extends Person; adds specialization, consultationFee, and available clinic days.  |

3. System Architecture & Object-Oriented Design (Continued)

| Class / Enum         | Role / OOP Principle | Key Methods & Responsibilities                                                                     |
|----------------------|----------------------|----------------------------------------------------------------------------------------------------|
| Appointment.java     | Domain Entity        | Links patient, doctor, date, timeSlot, status, and clinical visit outcome notes.                   |
| TriageSeverity.java  | Enumeration          | Defines LOW, MEDIUM, HIGH, CRITICAL urgency ranks (1-4) with clinical standing labels.             |
| StorageManager.java  | Persistence Engine   | Implements RFC 4180 CSV serialization with quote-escaping for patients, doctors, and appointments. |
| ClinicService.java   | Business Logic Layer | Triage priority sorting, conflict detection, map indexing, and billing invoice generation.         |
| ValidationUtils.java | Defensive Parser     | Regex validators, stream-safe line parsing, date checks, and terminal string truncation.           |
| Main.java            | Presentation / UI    | Interactive 10-option CLI menu loop, ASCII table renderers, and input flow orchestration.          |

3.1 Class Structural Hierarchy



4. Core Algorithms & Implementation Details

**1. Scheduling Conflict Avoidance Algorithm:** Before booking any appointment, ClinicService evaluates active appointments to prevent physician double-booking on identical date and time slots:

```
public boolean hasSchedulingConflict(int doctorId, String date, String timeSlot) {
    return appointmentMap.values().stream()
        .filter(a -> a.getDoctorId() == doctorId)
        .filter(a -> a.getStatus() == AppointmentStatus.SCHEDULED)
        .filter(a -> a.getAppointmentDate().equalsIgnoreCase(date.trim()))
        .anyMatch(a -> a.getTimeSlot().equalsIgnoreCase(timeSlot.trim()));
}
```

**2. Multi-Level Triage Queue Prioritization:** Java Streams dynamically sort patients by medical acuity (CRITICAL = Rank 1, HIGH = Rank 2, MEDIUM = Rank 3, LOW = Rank 4) with registration ID as tie-breaker:

```
public List<Patient> getTriagePriorityQueue() {
    return patientMap.values().stream()
        .sorted(Comparator.comparingInt((Patient p) -> p.getTriageSeverity().getPriorityRank())
            .thenComparing(Patient::getId))
        .collect(Collectors.toList());
}
```

5. Verification & Test Suite Matrix

| Test ID | Test Scenario       | Input Condition             | Expected Outcome                          | Status |
|---------|---------------------|-----------------------------|-------------------------------------------|--------|
| TC-01   | Register Patient    | Valid patient details       | Stored & persisted to patients.csv        | PASS   |
| TC-02   | Duplicate ID Check  | Existing patient ID 101     | Rejected with ID collision error          | PASS   |
| TC-03   | Triage Ordering     | Mixed severity patients     | CRITICAL sorted first ahead of LOW        | PASS   |
| TC-04   | Doctor Search       | Query: 'Cardio'             | Returns Cardiology specialists only       | PASS   |
| TC-05   | Conflict Booking    | Same doctor, date, slot     | Booking rejected; conflict prompt         | PASS   |
| TC-06   | Foreign Key Check   | Non-existent patient 999    | Booking rejected; patient not found       | PASS   |
| TC-07   | Invoice Calculation | Fee \$150 + Critical (\$50) | Total payable calculated as \$200.00      | PASS   |
| TC-08   | CSV Persistence     | Restart application         | All records reloaded from data/ directory | PASS   |
| TC-09   | EOF Termination     | Piped empty input stream    | Graceful shutdown; data synced            | PASS   |

6. Sample Execution Transcript

```
--- [3] Emergency Triage Priority Queue ---
+-----+-----+-----+-----+-----+
| ID | Name           | Blood | Triage Level | Urgency Clinical Standing |
+-----+-----+-----+-----+-----+
| 101 | Eleanor Vance   | O+    | CRITICAL    | Immediate Emergency       |
| 102 | Marcus Brody    | A+    | HIGH        | Urgent / High Priority    |
| 103 | Sophia Chen     | B+    | MEDIUM     | Moderate / Needs Attention|
| 104 | Lucas Miller    | AB-   | LOW         | Routine / Non-urgent     |
+-----+-----+-----+-----+-----+

--- [9] Clinical Billing Invoice ---
Invoice Ref: INV-00501 | Apt ID: #501 | Patient: Eleanor Vance (CRITICAL)
Doctor: Dr. Sarah Jenkins (Cardiology) | Fee: $150.00 | Urgency Charge: $50.00
TOTAL AMOUNT PAYABLE: $200.00 | Payment Status: PENDING
```

7. Future Scope & Conclusion

**Future Extensions:** Integration with relational database engines (PostgreSQL / SQLite) via JDBC, graphical patient appointment calendars via JavaFX, prescription management, and automated SMS appointment notifications.

**Conclusion:** The Healthcare Clinic & Patient Appointment System fulfills all requirements of the VITyarthi Build Your Own Project guidelines. It delivers 10 modular classes with clean inheritance hierarchies, multi-criteria stream triage sorting, robust CSV persistence, conflict-free appointment scheduling, and automated evaluation compatibility. Developed and submitted by **Krish Kumar (25BAI10528)**.